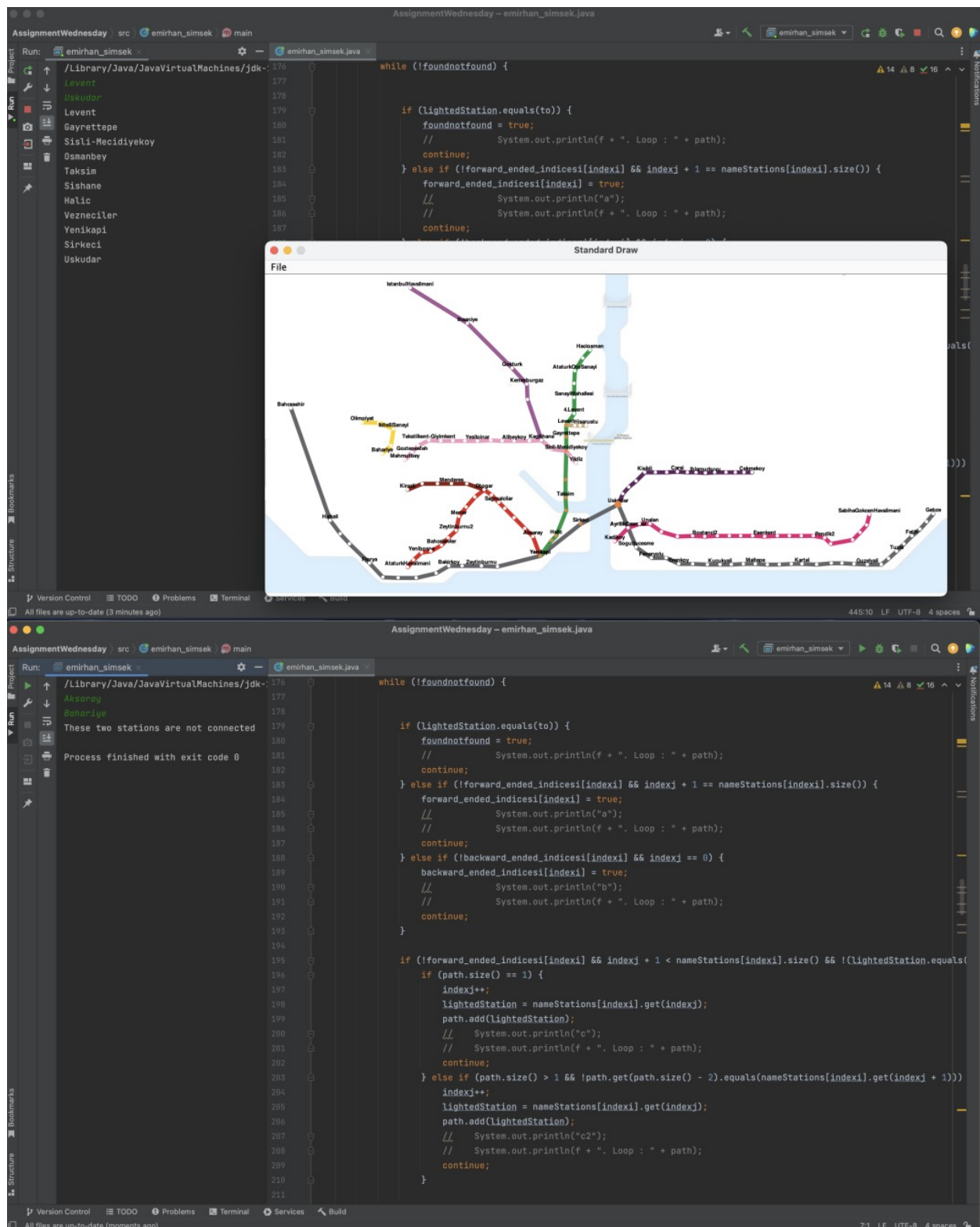


Firstly, read the text file line by line and place the inputs in different data structures to retrieve the data by index of the data structures (for java: arrays, arraylists, array of arraylists). For even numbered rows (0. 2. 4. etc.) the input is divided to two parts as the line names and line colors. Line names are first so all the first inputs in even numbered lines are added to the namelines data structure, and all the second inputs are added to the colors structure (first if even numbered second if odd numbered). Similarly for odd numbered rows, the input is divided to 3, even numbered elements are station names, odd elements are x coordinates and y coordinates.

Secondly, the algorithm for path finding begins. If the departure and destination location inputs are missing in the data structures the output is printed telling it is not found. If inputs exist, the algorithm is as follows: Find the destination location. Label it as lightedCity (we attain the station name to a variable) and add it to the path data structure. Enter a loop which repeats until a path is found or deliberately ended (by break method). Check if the labeled station is the destination. Check if the labeled station is the last or the first station of the line(that way we mark if the end of the line is reached or not). Then step forward or backward if the next or previous station is not in the path, the labeled city is not a breakpoint (cross section of lines), or the end of the line at that direction(forward backward) is not already reached(this is a condition since only way the end of the line is reached is that the route in that direction is already tried out in previous loop iterations and the algorithm should not try out the same path twice, it would be an endless loop). Stepping forward or backward in a line means changing labeled station 's second index by increasing or decreasing by one (for retrieving next station from the data structures) and adding the new labeled station to the path. After that, we check if the labeled station is a breaking point. If it is a breaking point, we check the possible steps and whether that route is already tried out similar as we did for non-breakpoints. This time we check the next or previous station in all of the crossing lines. We take the step if the end of that direction is not reached and the path we are building is not coming from that direction. The last steps is accepted and indices of the label is accordingly changed. The first index represents the lines and may change if the line is changed. These steps continue until end of a line, and if there is no steps to be made and no stations to be added, what is checked next in a loop iteration is if we can step back. This time we check if next or previous station is in the path. If the labeled station is a breakpoint we try out all the possible backsteps similar as before. We remove the labeled station from path and make the last element of the previous path the new labeled station. And if the labeled station is not a breakpoint, we check if the last station is in the path and check if the path size is 1. These are all possible steps to check all paths.

Thirdly, if path is not found, print message. If found, retrieve the coordinates of stations, and visualize the path on map. Enable commented codes to observe code's behavior better and see which iteration activates which part of the code and see which routes are tried out.

(We import libraries. Then initialize the terms, arrays, arraylists. They are all Java syntax. The explanation above is for the algorithm, see the terms, variables, data structures in the code)



```
AssignmentWednesday - emirhan_simsek.java

Run: emirhan_simsek
/Library/Java/JavaVirtualMachines/jdk-19.jdk/Contents/Home
Bogozici
Kadikoy
The station names provided are not present in this map.
Process finished with exit code 0

176 while (!foundnotfound) {
177
178
179     if (lightedStation.equals(to)) {
180         foundnotfound = true;
181         // System.out.println(f + ". Loop : " + path);
182         continue;
183     } else if (!forward_ended_indices[indexi] && indexj + 1 == nameStations[indexi].size())
184         forward_ended_indices[indexi] = true;
185         // System.out.println("a");
186         // System.out.println(f + ". Loop : " + path);
187         continue;
188     } else if (!backward_ended_indices[indexi] && indexj == 0) {
189         backward_ended_indices[indexi] = true;
190         // System.out.println("b");
191         // System.out.println(f + ". Loop : " + path);
192         continue;
193     }
194
195     if (!forward_ended_indices[indexi] && indexj + 1 < nameStations[indexi].size() && !lig
196         if (path.size() == 1) {
197             indexj++;
198             lightedStation = nameStations[indexi].get(indexj);
199             path.add(lightedStation);
200             // System.out.println("c");
201             // System.out.println(f + ". Loop : " + path);
202             continue;
203         } else if (path.size() > 1 && !path.get(path.size() - 2).equals(nameStations[indexi]
204             indexj++;
205             lightedStation = nameStations[indexi].get(indexj);
206             path.add(lightedStation);
207             // System.out.println("c2");
208             // System.out.println(f + ". Loop : " + path);
209             continue;
210         }
211     }
```

AssignmentWednesday - emirhan_simsek.java

Run: emirhan_simsek

/Library/Java/JavaVirtualMachines/jdk-19.jdk/Contents/Home

IstanbulHavalimani
SabihaGokcenHavalimani
IstanbulHavalimani
Ihsaniye
Gokturk
Kemerburgaz
Hasdal
Kagithane
Caglayan
Sisli-Mecidiyekoy
Osmanbey
Taksim
Sislihan
Halic
Vezneciler
Yenikapı
Sirkeci
Uskudar
AyrilikCesmesi
Acibadem
Unalan
Goztepe2
Yenisahra
Kozyatagi
Bostanci2
Kucukyali2
Maltepe2
Huzurevi
Gulsuyu
Esenkent
Hastane-Adliye
Soganlik
Kartal2
Yakacik
Pendik2

Standard Draw

176 while (!foundnotfound) {
177
178
179 if (lightedStation.equals(to)) {
180 foundnotfound = true;
181 // System.out.println(f + ". Loop : " + path);
182 continue;
183 } else if (!forward_ended_indices[indexi] && indexj + 1 == nameStations[indexi].size())
184 forward_ended_indices[indexi] = true;
185 // System.out.println("a");
186 // System.out.println(f + ". Loop : " + path);
187 continue;
188 } else if (!backward_ended_indices[indexi] && indexj == 0) {
189 backward_ended_indices[indexi] = true;
190 // System.out.println("b");
191 // System.out.println(f + ". Loop : " + path);
192 continue;
193 }
194
195 if (!forward_ended_indices[indexi] && indexj + 1 < nameStations[indexi].size() && !lig
196 if (path.size() == 1) {
197 indexj++;
198 lightedStation = nameStations[indexi].get(indexj);
199 path.add(lightedStation);
200 // System.out.println("c");
201 // System.out.println(f + ". Loop : " + path);
202 continue;
203 } else if (path.size() > 1 && !path.get(path.size() - 2).equals(nameStations[indexi]
204 indexj++;
205 lightedStation = nameStations[indexi].get(indexj);
206 path.add(lightedStation);
207 // System.out.println("c2");
208 // System.out.println(f + ". Loop : " + path);
209 continue;
210 }
211 }